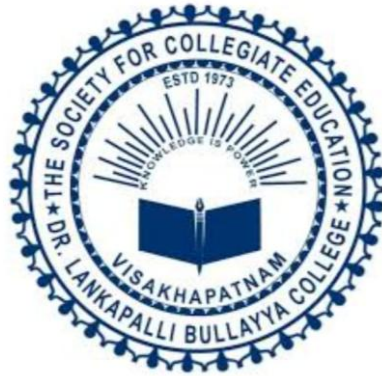


FAKE PRODUCT REVIEW DETECTION



ANDHRA UNIVERSITY
Dr. LANKAPALLI BULLAYYA COLLEGE
MCA PROJECT REPORT
ON
FAKE PRODECT REVIEW DETECTION

Submitted By: Nakka Deepika

Registration Number: 324228860064

Under the Guidance of: S. Satheeswara Reddy

Department: Department of Computer Applications

Academic Year: 2024-2026

INDEX

CONTENT

PAGE NO.

1. Introduction

1.1 Introduction to the Project	03
1.1.1 Scope.....	03
1.1.2 Problem Statement.....	04
1.2 Existing System	05
1.3 Proposed System including Modules.....	06

2. Requirement Analysis

2.1 Feasibility Study	10
2.1.1 Technical Feasibility.....	10
2.1.2 Economical Feasibility.....	10
2.1.2.1 Cost Benefit Analysis	10
2.1.3 Operational Feasibility.....	12
2.2 Software Requirement Specification	12
2.2.1 Functional Requirements	12
2.2.2 Non-Functional Requirements	14
2.2.3 Environment & Technology Requirements	16
2.2.3.1 Hardware Requirements.....	16
2.2.3.2 Software Requirements.....	16

3. Design

3.1 System Design	18
3.1.1 Introduction to UML.....	20
3.1.2 UML Diagrams	22
3.2 Interface Design	33
3.3 Database Design.....	35
3.3.1 Introduction to Backend.....	36
3.3.2 Normalization regarding Project Tables	37

4. Coding

4.1 Sample Code	44
-----------------------	----

5. Testing

5.1 Introduction to Testing.....	57
5.2 Test Cases	61

6. Screenshots

6.1 Input Screens.....	66
6.2 Output Screens.....	67

7. Conclusion	69
----------------------------	-----------

8. Bibliography	71
------------------------------	-----------

ACKNOWLEDGEMENT

I express my sincere gratitude to our Project Guide S.Satheeswara Reddy sir for their valuable guidance, continuous encouragement, throughout the development of the project title Fake Product Review Detection.

I would also like to extend my sincere thanks to the Head of the Department M.A. Prasad Sir and all the faculty members of the Department of Computer Science for providing the necessary facilities and academic support during the course of this project.

I am grateful to our Principal Dr. G.S.K. Chakravarty Sir.

My sincere thanks to CSC India and Prajna Resources Pvt. Ltd. for their valuable support and encouragement during the development of this project.

Abstract

Online shopping has become increasingly popular, making customer reviews an important factor in purchasing decisions. However, many e-commerce platforms contain fake or misleading reviews that can influence customers and reduce trust in online products. Detecting such fake reviews is a challenging task due to the large volume of user-generated content.

The Fake Product Review Detection system is developed to identify whether a product review is fake or genuine using machine learning techniques. The system accepts product review text as input, preprocesses the text by removing unnecessary words and symbols, extracts useful features, and applies a trained machine learning model to classify the review. The prediction result is then displayed to the user and stored in the database for future reference.

This project improves the reliability of online shopping by helping customers identify misleading reviews and make informed purchasing decisions. It also assists e-commerce platforms in maintaining the authenticity of product reviews. The proposed system is simple, accurate, user-friendly, and can be further enhanced by incorporating advanced Natural Language Processing (NLP) techniques and deep learning models for improved performance.

1. INTRODUCTION

1.1 Introduction to the project

In the modern era of e-commerce, online customer reviews play a significant role in influencing purchasing decisions. Before buying a product, most customers rely on reviews and ratings to evaluate its quality, performance, and reliability. However, the increasing popularity of online shopping has also led to the rise of fake product reviews, which are intentionally written to mislead customers by promoting or damaging a product's reputation.

Fake reviews can negatively affect both consumers and businesses. They may cause customers to purchase low-quality products based on false positive reviews or avoid good products due to fake negative reviews. As a result, identifying and filtering fake reviews has become an important challenge for e-commerce platforms.

The objective of this project is to develop a Fake Product Review Detection system using Machine Learning and Natural Language Processing (NLP). The system analyzes the textual content of product reviews and classifies them as genuine or fake based on learned patterns and linguistic features. By using a labeled dataset and suitable machine learning algorithms, the model aims to improve the accuracy of fake review detection.

This project helps promote transparency and trust in online shopping by assisting users and e-commerce platforms in identifying misleading reviews. It also demonstrates the practical application of Artificial Intelligence in solving real-world problems related to online consumer behavior.

.1.1 Scope

1. Detection of Fake Reviews

The project focuses on identifying whether a product review is genuine or fake by analyzing the review text using Machine Learning and Natural Language Processing (NLP) techniques.

2. Support for E-commerce Platforms

The system can assist e-commerce websites in filtering misleading reviews, improving the reliability and credibility of their review systems.

3. Customer Decision Support

By detecting fake reviews, the project helps customers make informed purchasing decisions based on authentic and trustworthy feedback.

4. Machine Learning-Based Classification

The project applies machine learning algorithms to classify reviews into genuine or fake categories, enabling automated and efficient review analysis.

5. Text Analysis and Feature Extraction

The system analyzes review content by extracting meaningful textual features such as word frequency, sentiment, and writing patterns to improve detection accuracy.

6. Scalability and Future Enhancement

The proposed system can be extended to support multiple languages, detect spam reviews in real time, and integrate with large-scale e-commerce platforms using advanced Deep Learning models.

7. Real-World Applications

The project can be used in online marketplaces, review websites, travel booking platforms, restaurant review portals, and mobile shopping applications to enhance trust and reduce fraudulent activities.

.1.2 Problem Statement

Background

Online shopping platforms have become one of the primary sources for purchasing products. Customers depend on product reviews and ratings to evaluate the quality and reliability of products before making a purchase.

Existing Problem

The rapid growth of e-commerce has led to an increase in fake product reviews. These reviews are intentionally written to mislead customers by promoting or criticizing products unfairly. As a result, customers may make incorrect purchasing decisions, and businesses may suffer reputational and financial losses.

Need for the Project

Manual identification of fake reviews is difficult, time-consuming, and ineffective due to the large number of reviews posted every day. Therefore, an automated system is required to accurately identify fake reviews and improve the trustworthiness of online review platforms.

Proposed Solution

The proposed Fake Product Review Detection system uses Machine Learning and Natural Language Processing (NLP) techniques to analyze review text and classify reviews as genuine or fake. The system helps improve the reliability of online reviews and supports customers in making informed purchasing decision

Expected Outcome

The project aims to reduce the impact of fake reviews, increase customer trust in ecommerce platforms, and provide an efficient and scalable solution for automatic review classification.

2.Existing System

Overview

Most existing e-commerce platforms allow customers to post product reviews and ratings without fully verifying their authenticity. These reviews are displayed to help other customers make purchasing decisions.

Working of the Existing System

The existing system mainly relies on user-submitted reviews and basic moderation techniques. Some platforms use simple spam filters or manual verification, but these methods are not sufficient to detect all fake reviews.

Limitations

- Manual verification is time-consuming and requires significant human effort.
- Traditional spam filters cannot accurately identify sophisticated fake reviews.
- Fake positive and fake negative reviews can easily influence product ratings.
- Large volumes of reviews make continuous monitoring difficult.
- Customers may lose trust in the platform due to misleading reviews.

Need for Improvement

An intelligent and automated review detection system is required to accurately identify fake reviews using Machine Learning and Natural Language Processing (NLP). Such a system can improve review quality, enhance customer trust, and support fair business practices.

2. Proposed System

Overview

The proposed system is designed to automatically detect fake product reviews using Machine Learning and Natural Language Processing (NLP). It analyzes the review text and classifies each review as either genuine or fake.

Working of the Proposed System

The system collects product reviews, preprocesses the text by removing unnecessary words and symbols, extracts important textual features, and applies a trained machine learning model to classify the reviews. The final result indicates whether a review is genuine or fake.

Advantages

- Automatically detects fake product reviews with high accuracy.
- Reduces manual effort and saves time.
- Improves the reliability and credibility of online reviews.
- Helps customers make informed purchasing decisions.
- Supports e-commerce platforms in maintaining trustworthy review systems.

Technologies Used

- Machine Learning (ML)
- Natural Language Processing (NLP)
- Python
- Data Preprocessing Techniques
- Classification Algorithms (such as Logistic Regression, Naïve Bayes, or Random Forest)

Future Enhancements

- Support for multilingual review detection.
- Real-time fake review detection on e-commerce platforms.
- Integration with deep learning models for improved accuracy.
- Detection of spam reviews based on user behavior and review patterns.

Modules

1. Data Collection Module

This module collects product review data from the dataset.

2.Data Preprocessing Module

In this module, the review text is cleaned by removing punctuation, special characters, stop words, and unnecessary spaces. The text is then converted into a suitable format for analysis.

3. Feature Extraction Module

This module extracts important features from the review text using Natural Language Processing (NLP) techniques such as TF-IDF or Bag of Words. These features help the model understand the content of the reviews.

4. Model Training Module

The extracted features are used to train a machine learning classification model. The model learns patterns from the labeled data to distinguish between genuine and fake reviews.

5. Review Classification Module

This module accepts a new product review as input and uses the trained model to classify it as either Genuine or Fake.

6. Result and Evaluation Module

The final module displays the classification result and evaluates the model's performance using metrics such as accuracy, precision, recall, and F1-score.

2. REQUIREMENT ANALYSIS

2.1 Feasibility Study

A feasibility study is conducted to determine whether the proposed Fake Product Review Detection system is practical, cost-effective, and technically achievable. It evaluates different aspects of the project before development.

2.1.1 Technical Feasibility

The proposed system is technically feasible because it uses widely available technologies such as Python, Machine Learning, and Natural Language Processing (NLP).

□ Schedule Feasibility

The project can be completed within the planned academic timeline. The development phases, including data collection, preprocessing, model training, testing, and evaluation, can be completed within the allocated project duration.

2.1.2 Economical Feasibility

Economic feasibility evaluates whether the proposed system can be developed and maintained within a reasonable budget. The Fake Product Review Detection project is economically feasible because it primarily uses open-source technologies such as Python, Machine Learning libraries, and Natural Language Processing (NLP) tools, which are available free of cost. The project does not require expensive hardware or software licenses and can be developed using a standard computer. As a result, the overall development, implementation, and maintenance costs are low, making the system cost effective and suitable for academic as well as practical applications.

2.1.2.1 Cost–Benefit Analysis

Cost–Benefit Analysis is used to evaluate whether the benefits of the proposed system outweigh the costs involved in its development and implementation. The Fake Product Review Detection system is developed using open-source technologies, which minimizes development costs while providing significant benefits to users and e-commerce platforms.

Cost Analysis

Cost Component	Description	Estimated Cost
Hardware	Standard computer/laptop for development	Existing Resource
Software	Python, Jupyter Notebook, VS Code, ML & NLP Libraries	Free (Open Source)
Dataset	Publicly available review datasets	Free
Internet	Downloading datasets and libraries	Minimal
Maintenance	Model updates and system improvements	Low

Benefit Analysis

Benefit	Description
Improved Review Accuracy	Identifies fake reviews with better accuracy using Machine Learning.
Customer Trust	Helps customers rely on genuine product reviews.
Better Purchasing Decisions	Reduces the chances of buying products based on misleading reviews.
Reduced Manual Effort	Automates the review verification process.
Support for E-commerce Platforms	Improves the credibility and reliability of online review systems.
Scalability	Can be extended to support larger datasets and multiple languages.

The proposed Fake Product Review Detection system offers greater benefits than the costs involved. Since the project uses free software tools and publicly available datasets, the implementation cost is very low. The system improves customer trust, enhances the reliability of online reviews, and provides a cost-effective solution for detecting fake product reviews.

2.1.3 Operational Feasibility

The system is easy to use and can be integrated into e-commerce platforms to automatically detect fake product reviews. It improves the reliability of online reviews and helps customers make informed purchasing decisions.

2.2 Software Requirement Specification

2.2.1 Functional Requirements

2.2.1.1 Input Requirements

1. Product Review Text

The primary input to the system is the product review text submitted by a customer. The review contains the opinions and experiences of the user about a product.

2. Review Dataset

A labeled dataset containing both genuine and fake product reviews is required for training and testing the machine learning model. The dataset helps the model learn patterns for accurate classification.

3. User Review Input

The system accepts a new product review entered by the user. This review is analyzed to determine whether it is genuine or fake.

4. Preprocessed Text

The input review is cleaned by removing punctuation, special characters, stop words, and unnecessary spaces. The processed text is then prepared for feature extraction.

5. Feature Extraction Input

The cleaned review text is converted into numerical feature vectors using techniques such as TF-IDF or Bag of Words. These features are provided as input to the machine learning model for classification.

2.2.1.2 Output Requirements

1. Review Classification Result

The system displays the final classification result by identifying whether the submitted product review is Genuine or Fake.

2. Prediction Accuracy

The system provides the prediction based on the trained machine learning model, ensuring accurate classification of product reviews.

3. Review Status

The output clearly indicates the status of the review, allowing users or administrators to determine whether the review is trustworthy.

4. User-Friendly Display

The classification result is presented in a simple and easy-to-understand format so that users can quickly interpret the output.

5. Model Performance Metrics

The system displays evaluation metrics such as Accuracy, Precision, Recall, and F1-Score to measure the performance of the machine learning model.

2.2.1.3 Computational Requirements

1. Processor

The system requires a standard computer with an Intel Core i3/i5 processor or an equivalent processor to perform data preprocessing, model training, and review classification efficiently.

2. Memory (RAM)

A minimum of 4 GB RAM is required for basic execution. However, 8 GB RAM is recommended for faster processing and better performance while handling large datasets.

3. Storage

At least 10 GB of free disk space is required to store the dataset, machine learning libraries, project files, and trained models.

4. Operating System

The system can be developed and executed on Windows 10/11, Linux, or macOS, as Python and the required libraries support multiple operating systems.

5. Software Requirements

The project requires Python, Jupyter Notebook or Visual Studio Code, along with machine learning and NLP libraries such as NumPy, Pandas, Scikit-learn, NLTK, and Matplotlib.

6. Internet Connectivity

An internet connection is required for downloading datasets, installing Python libraries, and updating software packages. Once the setup is complete, the system can perform review detection offline.

7. Performance Requirements

The system should process review text efficiently and classify reviews with good accuracy while maintaining low response time and effective resource utilization.

2.2.1.4 Storage Requirements

The Fake Product Review Detection system requires a well-structured storage mechanism to manage and maintain data efficiently. The database is used to store product reviews that are collected for training and testing the machine learning model, along with their corresponding labels such as genuine or fake. It also stores user-submitted reviews and their prediction results for future reference and analysis. In addition, the trained machine learning model and extracted features are stored so that the system can directly use them for prediction without retraining every time. The database ensures data security by preventing unauthorized access.

2.2.2 Non-Functional Requirements

2.2.2.1 Quality Metrics

Quality Metrics are the standards used to evaluate the overall quality and performance of a software system. They help determine whether the system is reliable, efficient, accurate, and suitable for its intended purpose. In the Fake Product Review Detection project, quality metrics ensure that the system produces correct results, responds quickly, and provides a good user experience.

Reliability

The system should consistently provide accurate results in detecting fake and genuine product reviews under different input conditions and workloads without failures.

Usability

The system should be user-friendly with a simple interface, allowing users to easily input reviews and understand whether a review is genuine or fake without technical difficulty.

Response Time

The system should generate predictions quickly after receiving the input review, ensuring minimal delay and efficient processing of data.

Throughput

The system should be capable of processing a large number of product reviews efficiently within a given time period without affecting performance.

Availability

The system should remain operational and accessible whenever required, ensuring minimal downtime and continuous service for users.

Maintainability

The system should be easy to maintain and update, allowing improvements in algorithms, datasets, and functionality without major changes to the overall structure

2.2.3 Environment & Technology Requirements

2.2.3.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	Minimum 4 GB (8 GB recommended)
Hard Disk	256 GB or higher
System Type	64-bit Computer
Input Devices	Keyboard, Mouse
Output Device	Monitor
Internet	Required

2.2.3.2 Software Requirements

Software	Description
Operating System	Windows 10 / Linux
Programming Language	Python
IDE / Tool	VS Code / PyCharm
Libraries	NumPy, Pandas, Scikit-learn
Framework	Machine Learning
Database (Optional)	MySQL
Browser	Google Chrome

3. Design

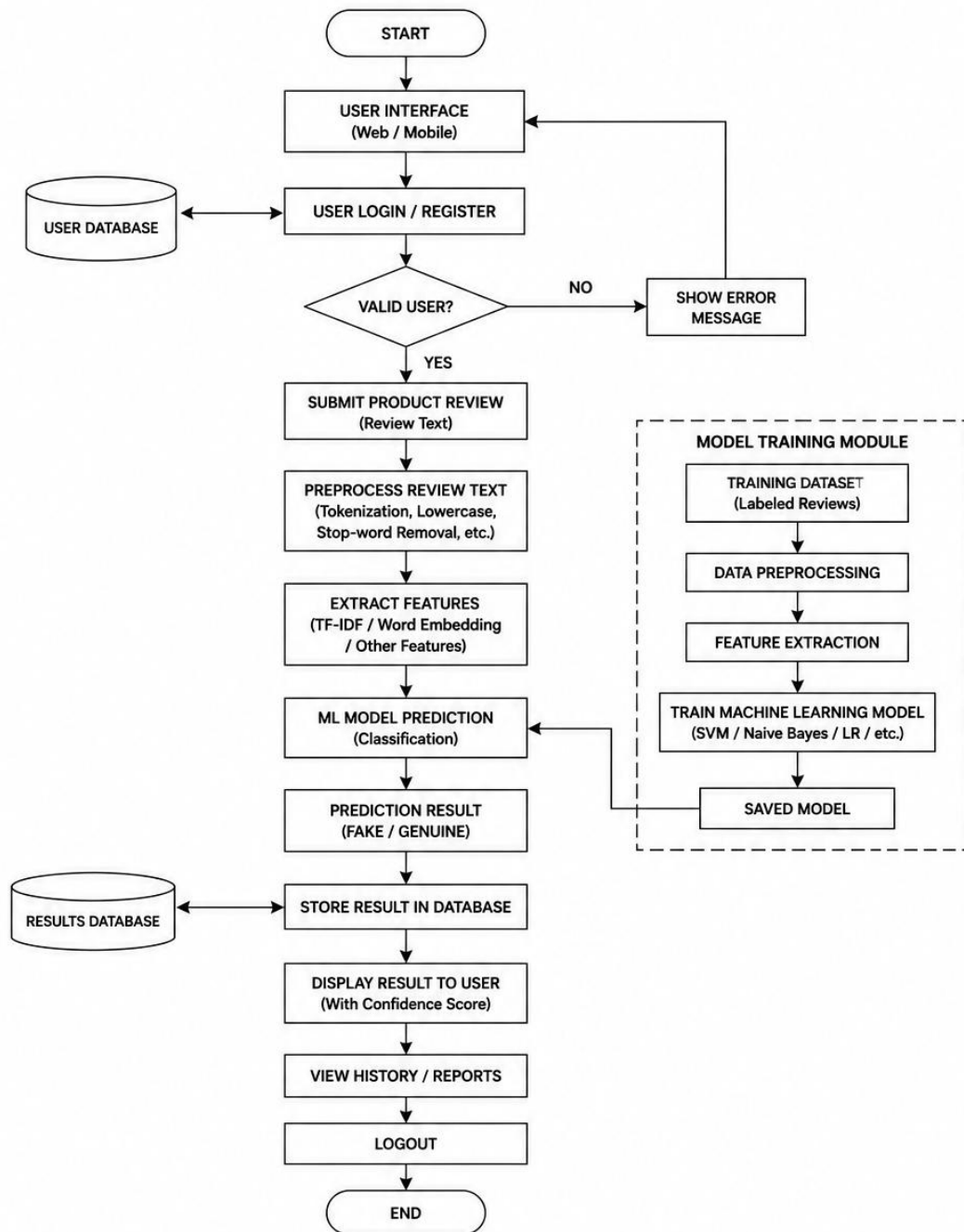
3.1 System Design

System Design is the process of defining the architecture, components, modules, interfaces, and data flow of a software system before implementation. It provides a clear blueprint for developing the application and ensures that all system requirements are met efficiently.

In the Fake Product Review Detection system, the design focuses on collecting product reviews, preprocessing the review data, extracting useful features, and applying a machine learning model to classify reviews as either **Fake** or **Genuine**. The system is designed to process user input accurately and display the prediction results in a simple and user-friendly manner.

The system consists of several modules, including data collection, data preprocessing, feature extraction, model training, review classification, and result generation. Each module performs a specific task and works together to ensure accurate detection of fake product reviews. The modular design makes the system easier to understand, maintain, and upgrade.

SYSTEM DESIGN FLOWCHART – FAKE PRODUCT REVIEW DETECTION SYSTEM



FLOW DESCRIPTION

1. User logs in or registers in the system.
2. User submits a product review.
3. The system preprocesses the review text.
4. Features are extracted from the text.
5. The trained ML model predicts whether the review is Fake or Genuine.
6. The result is stored in the database.
7. The result is displayed to the user.
8. User can view history/reports and logout.

SYSTEM COMPONENTS

- User Interface
- User Database
- Preprocessing Module
- Feature Extraction Module
- Machine Learning Model
- Results Database
- Model Training Module

DESCRIPTION

The system allows users to submit product reviews. The review goes through preprocessing and feature extraction. The machine learning model classifies the review as Fake or Genuine. The result is stored in the database and displayed to the user. The model is trained separately using labeled data.

3.1.1 Introduction to UML

Unified Modeling Language (UML) is a standard visual language used to design and represent software systems. It helps developers and stakeholders understand the structure and behavior of a system before actual coding begins. UML uses diagrams to show how different parts of a system interact with each other.

UML is widely used in software engineering to visualize system architecture, plan system design, and document software requirements. It makes complex systems easier to understand by presenting them in a graphical form. By using UML diagrams, developers can identify system components, relationships, and workflows clearly.

UML supports object-oriented concepts such as classes, objects, inheritance, and relationships. It provides different types of diagrams, including use case diagrams, class diagrams, sequence diagrams, and activity diagrams, each serving a specific purpose in system modeling.

Overall, UML improves communication among team members, reduces development errors, and helps in building well-structured and efficient software systems.

UML provides different types of diagrams to represent the structure and behavior of a software system. Each diagram has a specific purpose and helps developers understand different aspects of the system.

1. Use Case Diagram

A Use Case Diagram shows the interaction between users (actors) and the system. It represents the main functions of the system and helps identify the system requirements from the user's perspective.

2. Class Diagram

A Class Diagram represents the static structure of the system. It shows the classes, their attributes, methods, and the relationships between them. It is used to design the overall architecture of the application.

3. Sequence Diagram

A Sequence Diagram illustrates how objects interact with each other in a specific sequence over time. It shows the flow of messages exchanged between different components to perform a particular task.

4. Activity Diagram

An Activity Diagram represents the workflow of a process or operation. It shows the sequence of activities, decisions, and actions involved in completing a task within the system.

5. State Diagram

A State Diagram describes the different states of an object and the transitions between those states during its lifecycle. It helps model the behavior of an object based on different events.

6. Component Diagram

A Component Diagram shows the different software components of the system and how they are connected. It helps represent the modular structure of the application.

7. Deployment Diagram

A Deployment Diagram illustrates how the software is deployed on hardware devices. It shows the physical architecture of the system, including servers, computers, and network connections.

These UML diagrams help developers visualize, design, document, and maintain software systems effectively. They improve communication among team members and make software development more organized and efficient.

3.1.2 UML diagrams

3.1.2.1 Scenarios

Use Case Diagram Scenario

A user enters a product review into the system. The system processes the review and predicts whether it is fake or genuine. The prediction result is then displayed to the user.

Class Diagram Scenario

The system manages users, product reviews, datasets, machine learning models, and prediction results. These classes interact to process reviews and generate accurate predictions.

Sequence Diagram Scenario

The user submits a review. The system preprocesses the review, extracts features, sends the data to the machine learning model, receives the prediction, and displays the result.

Activity Diagram Scenario

The process starts when a review is entered. The review is preprocessed, features are extracted, the model classifies the review, and the final result is displayed.

State Diagram Scenario

A review moves through different states: Submitted, Preprocessed, Feature Extracted, Classified, Fake/Genuine, and Result Displayed.

Component Diagram Scenario

The user interface sends the review to the preprocessing module, which forwards it to the feature extraction module and machine learning model. The prediction result is then returned to the user interface.

Deployment Diagram Scenario

The user accesses the application through a computer or web browser. The application communicates with the machine learning model and database to classify reviews and display the prediction.

3.1.2.2 Use-Case Diagrams

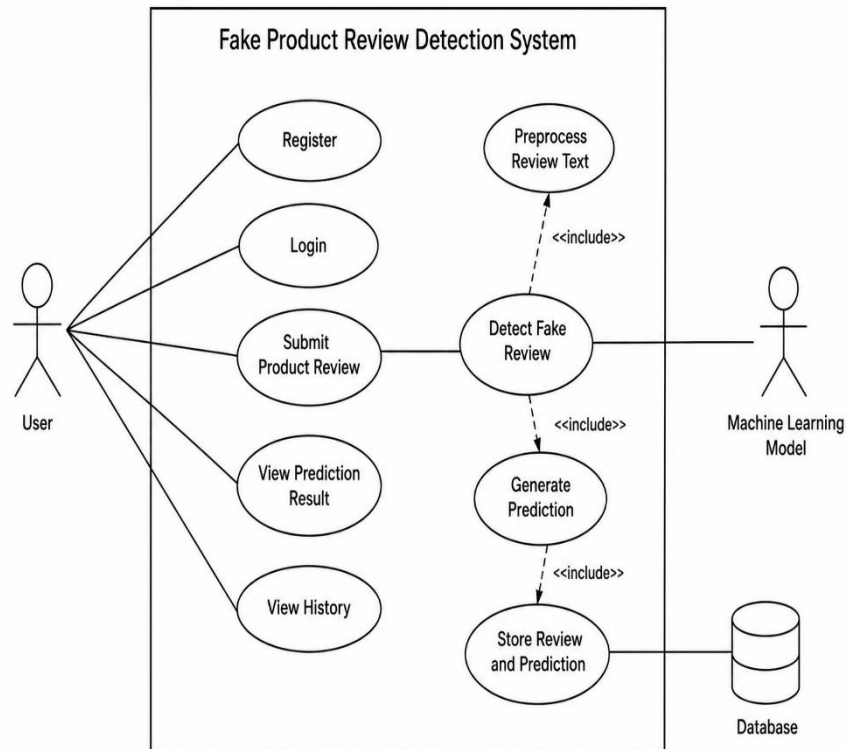
Actor

Primary Actor: User

The user is the main actor who interacts with the Fake Product Review Detection System. The user can register, log in, submit product reviews, and view the prediction results generated by the system.

Use Cases:

- Register
- Login
- Submit Product Review
- Detect Fake Review
- View Prediction Result
- View History
- Store Review and Prediction



Description

The Use Case Diagram represents the interaction between the user and the Fake Product Review Detection System. Initially, the user registers and logs into the system. After successful authentication, the user submits a product review. The system preprocesses the review, extracts important features, and applies the machine learning model to classify the review as either Fake or Genuine. The prediction result is then displayed to the user and stored in the database for future reference. The user can also view previously analyzed reviews and prediction history. This diagram provides a high-level overview of the system's functionality and the interactions between the user and the system.

Class Diagram:

Actor

Primary Actor: User

The user is the primary actor who interacts with the Fake Product Review Detection System. The user can register, log in, submit product reviews, and view the prediction results generated by the system.

Classes

- User
- Review
- Review Processor
- Feature Extractor
- Fake Review Model
- Prediction
- Database

CLASS DIAGRAM

Actor

User

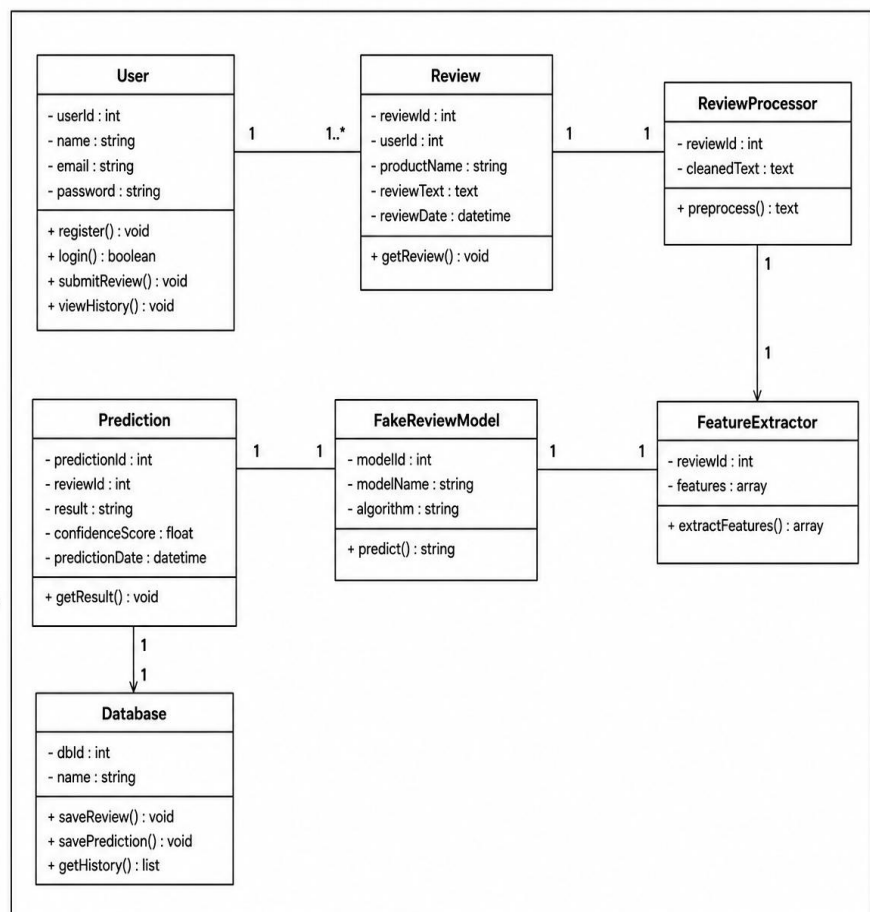
Classes

1. User
2. Review
3. ReviewProcessor
4. FeatureExtractor
5. FakeReviewModel
6. Prediction
7. Database

Description

The Class Diagram represents the structure of the Fake Product Review Detection System. The User class contains user details and can submit multiple reviews. The Review class stores the review text along with the date and is associated with the user. The ReviewProcessor class preprocesses the review text, which is then passed to the FeatureExtractor class for feature extraction.

The extracted features are given to the FakeReviewModel class, which uses a machine learning algorithm to predict whether the review is fake or genuine. The prediction result is stored in the Prediction class and also saved in the Database class for future reference.



Description

The Class Diagram represents the static structure of the Fake Product Review Detection System. It illustrates the classes, their attributes, methods, and the relationships between them. The **User** class manages user registration, login, and review submission. The Review class stores the review details entered by the user.

The Review Processor class preprocesses the review text by removing unwanted characters and preparing the data for analysis. The Feature Extractor class extracts important textual features from the processed review. These features are then passed to the Fake Review Model, which applies a machine learning algorithm to classify the review as Fake or Genuine.

The Prediction class stores the classification result along with the confidence score. Finally, the Database class stores user information, product reviews, and prediction results for future retrieval. The relationships among these classes ensure efficient communication, data management, and accurate review classification throughout the system

Object Diagram

Actor

Primary Actor: User

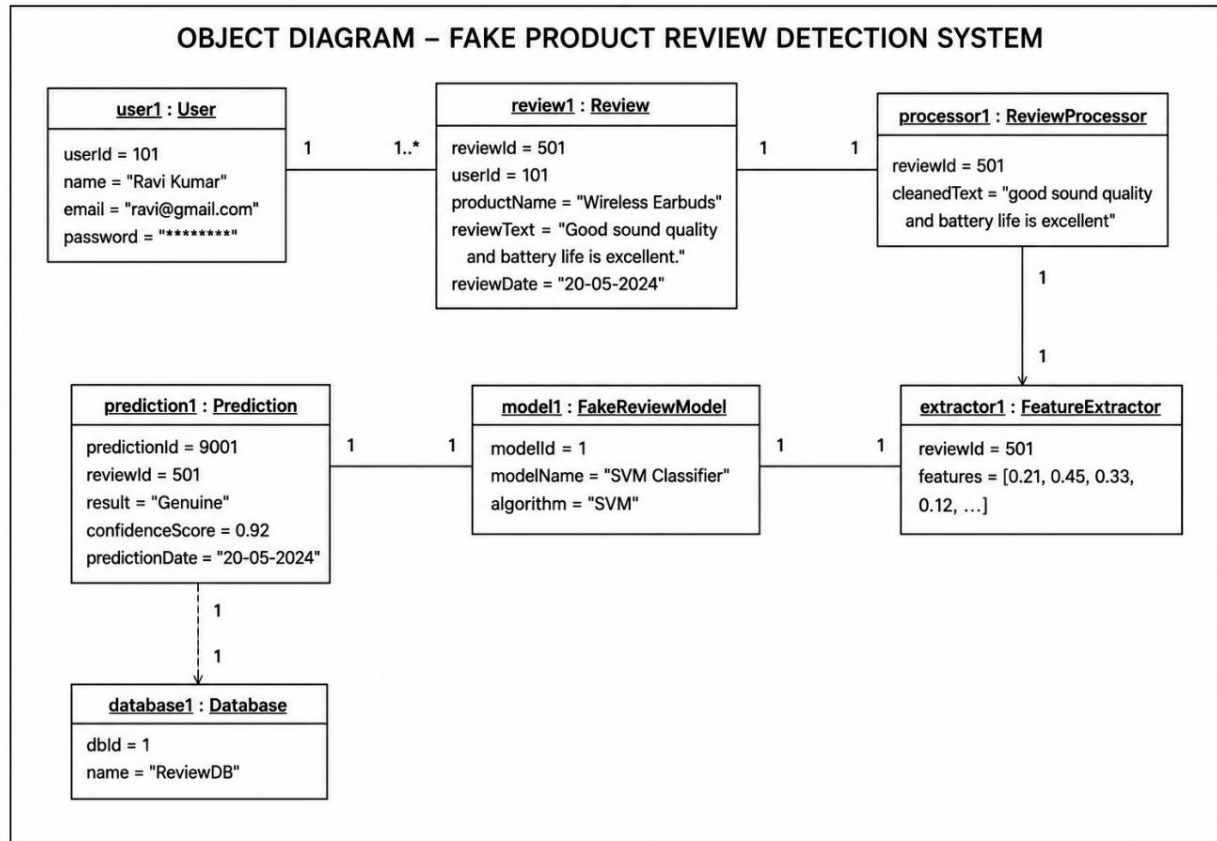
Object Diagram

The user is the primary actor who interacts with the Fake Product Review Detection System. The user submits product reviews and receives the prediction results generated by the system.

Objects

- User Object
- Review Object
- Review Processor Object
- Feature Extractor Object

- Fake Review Model Object
- Prediction Object
- Database Object



Description

The Object Diagram represents the instances of the classes and their relationships at a specific point in time. It provides a snapshot of how the objects interact during the execution of the Fake Product Review Detection System. The User object submits a product review, which is stored in the Review object. The Review Processor object preprocesses the review text and forwards it to the Feature Extractor object. The extracted features are passed to the Fake Review Model object, which predicts whether the review is Fake or Genuine. The prediction is stored in the Prediction object and saved in the Database object. This diagram helps in understanding the real-time interaction among objects during review analysis.

Sequence Diagram

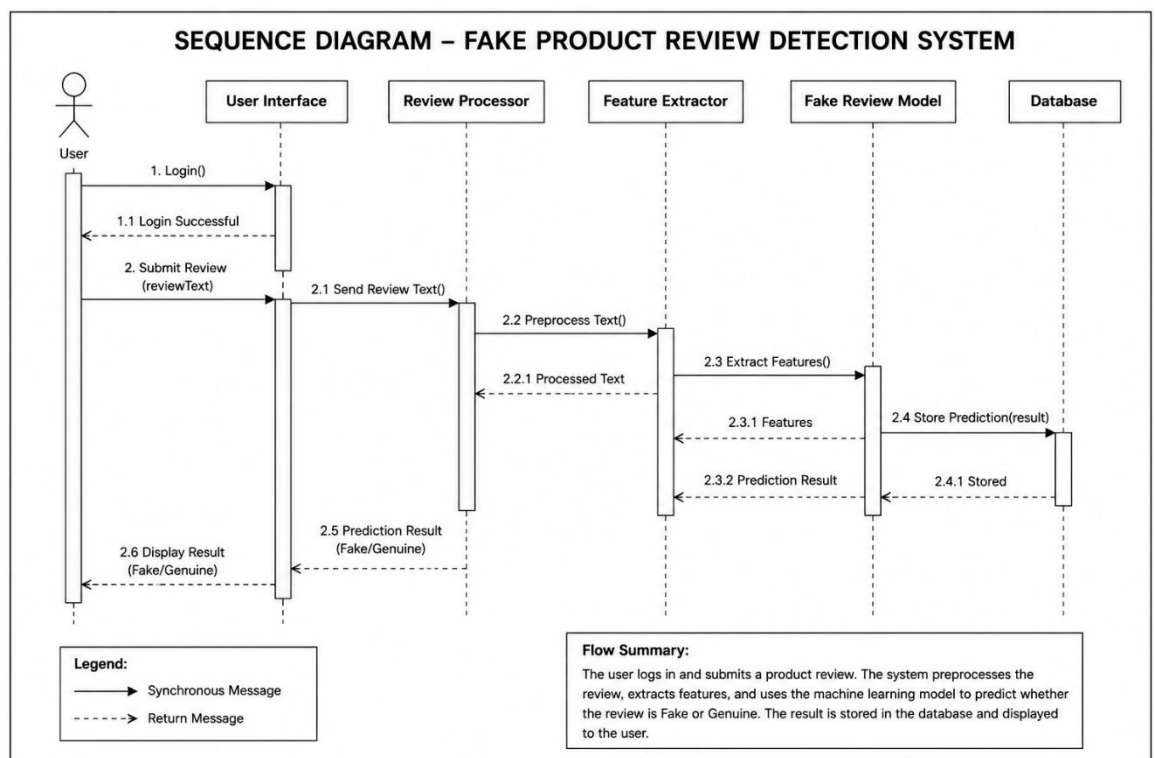
Actor

Primary Actor: User

The user initiates the sequence by interacting with the Fake Product Review Detection System.

Objects

- User
- User Interface
- Review Processor
- Feature Extractor
- Fake Review Model
- Database
- Prediction Result



Description

The Sequence Diagram illustrates the chronological flow of messages exchanged between the system objects. The process begins when the User logs into the application and submits a product review through the User Interface. The review is sent to the Review Processor, where preprocessing is performed. The processed review is then forwarded to the Feature Extractor, which extracts important features from the review text. These features are passed to the Fake Review Model, which analyzes the review and predicts whether it is Fake or Genuine. The prediction result is stored in **the** Database and finally displayed to the user through the user interface. The Sequence Diagram clearly shows the order of interactions between the system components from the beginning to the end of the review detection process.

Activity Diagram

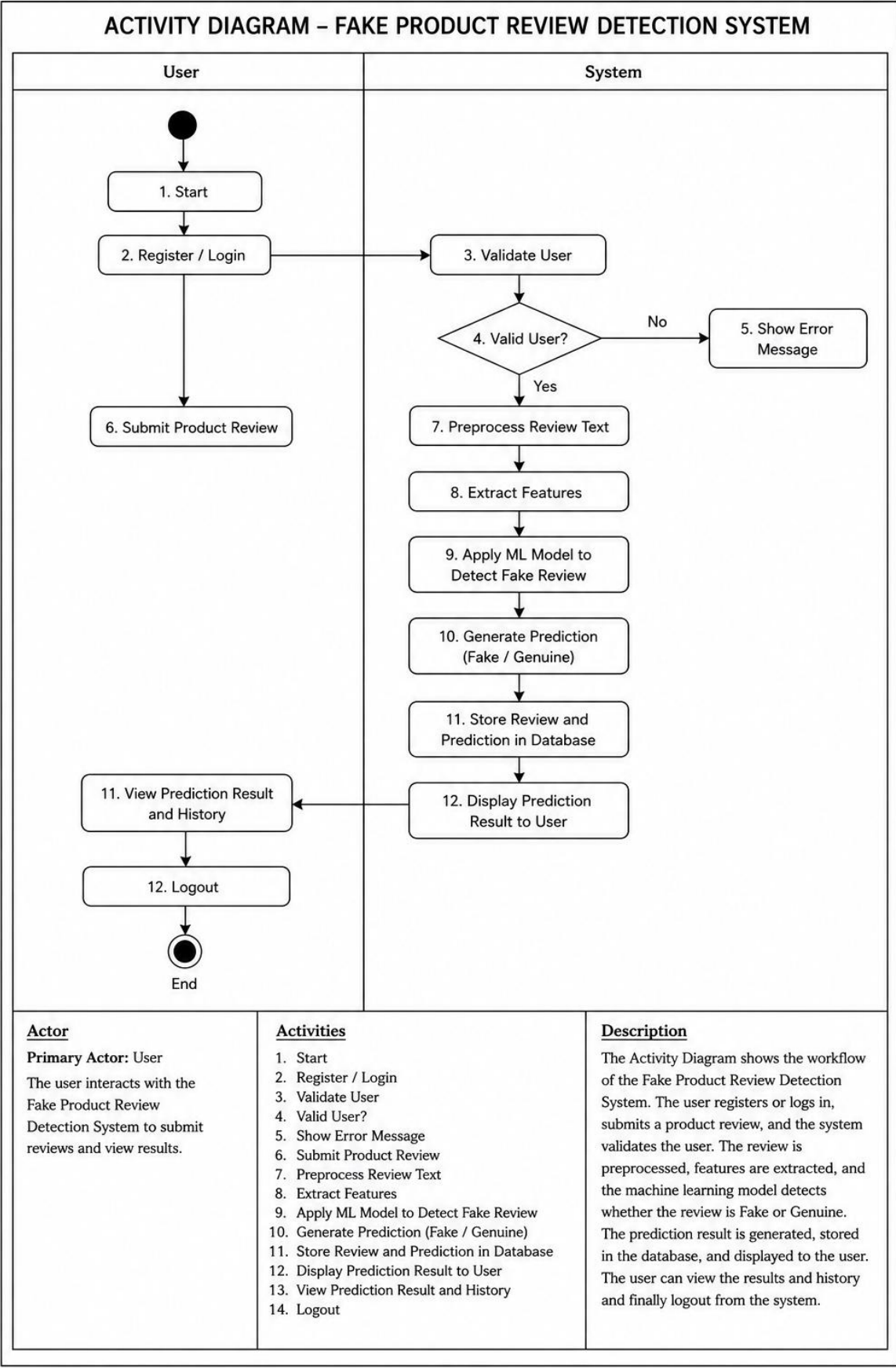
Actor

Primary Actor: User

The user is the primary actor who interacts with the Fake Product Review Detection System. The user performs activities such as registration, login, submitting product reviews, and viewing prediction results.

Activities

- Register
- Login
- Submit Product Review
- Preprocess Review
- Extract Features
- Detect Fake Review
- Generate Prediction
- Store Result in Database
- Display Prediction Result
- Logout



Description

The Activity Diagram illustrates the workflow of the Fake Product Review Detection System from the beginning to the end of the process. The user first registers or logs into the system. After successful authentication, the user submits a product review. The system preprocesses the review text, extracts important features, and applies the machine learning model to classify the review as Fake or Genuine. The prediction result is generated, stored in the database, and displayed to the user. Finally, the user can view the prediction result and log out of the system. The Activity Diagram helps in understanding the sequence of activities performed by the system during review analysis.

State Diagram

Actor

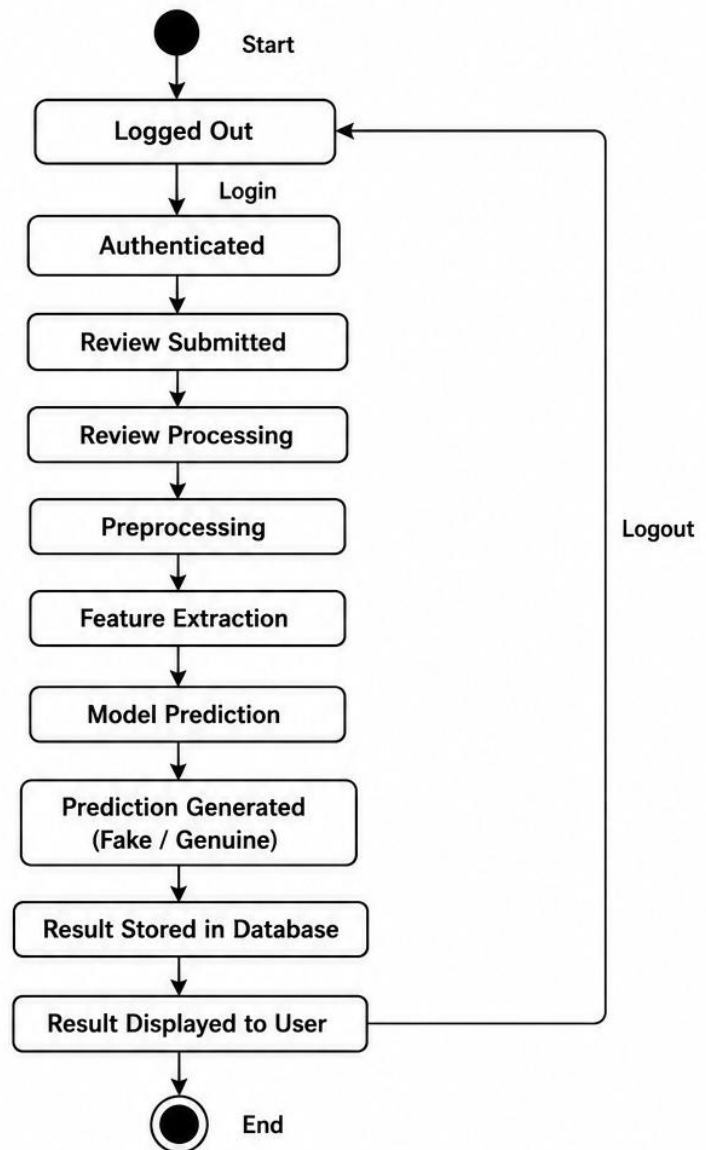
Primary Actor: User

The user changes the state of the Fake Product Review Detection System by performing actions such as logging in, submitting reviews, and logging out.

States

- Start
- Logged Out
- Authenticated
- Review Submitted
- Review Processing
- Prediction Generated
- Result Stored
- Result Displayed
- Logout
- End

STATE DIAGRAM – FAKE PRODUCT REVIEW DETECTION SYSTEM



ACTOR

Primary Actor: User

The user logs in, submits reviews, views prediction results, and logs out.

STATES

- Start
- Logged Out
- Authenticated
- Review Submitted
- Review Processing
- Preprocessing
- Feature Extraction
- Model Prediction
- Prediction Generated (Fake / Genuine)
- Result Stored in Database
- Result Displayed to User
- End

DESCRIPTION

The State Diagram illustrates the different states of the Fake Product Review Detection System from login to review submission, machine learning prediction, result display, and logout. The transitions show how the system changes state based on user actions and internal processing.

Description

The State Diagram represents the different states through which the Fake Product Review Detection System passes during its execution. The system starts in the Logged out state. After successful login, it moves to the Authenticated state. When the user submits a review, the system enters the Review Processing state, where preprocessing, feature extraction, and machine learning prediction are performed. After the prediction is generated, the result is stored in the database and displayed to the user. Finally, when the user logs out, the system returns to the Logged Out state and reaches the end of the process. The State Diagram illustrates how the system changes from one state to another based on user actions and internal processing.

3.2 Interface Design

Introduction

The input interface is designed to allow users to enter product review information easily and accurately. A simple and user-friendly interface improves the overall usability of the system and reduces input errors. The Fake Product Review Detection system accepts review details from the user and processes them to determine whether the review is fake or genuine.

Input Form 1: User Login

Purpose:

Allows registered users to log in to the system.

Input Fields:

- Username
- Password

Buttons:

- Login

- Reset

Input Form 2: User Registration

Purpose:

Allows new users to create an account.

Input Fields:

- Full Name
- Email Address
- Username
- Password
- Confirm Password

Buttons:

- Register
- Clear

Input Form 3: Product Review Submission

Purpose:

Allows users to submit a product review for analysis.

Input Fields:

- Product ID
- Product Name
- Product Category
- Review Title
- Review Text

Buttons:

- Submit Review

- Clear

Input Form 4: Review Prediction

Purpose:

Displays the prediction result after analyzing the review.

Output Display:

- Review Entered
- Prediction Result (Fake/Genuine)
- Confidence Score (Optional)

Buttons:

- Analyze Another Review
- Logout

Description

The input forms are designed with validation to ensure that users provide complete and valid information before processing. Mandatory fields must be filled before submission. The system accepts the review text, preprocesses it, applies the machine learning model, and displays the prediction result in an easy-to-understand format.

3.3 Database Design

Database design is the process of organizing and structuring data so that it can be stored, retrieved, and managed efficiently. A well-designed database improves data integrity, reduces redundancy, and ensures accurate processing of information. It provides a logical structure for storing user details, product reviews, prediction results, and other system data.

In the Fake Product Review Detection system, the database is designed to store information related to users, product reviews, and prediction results. Each review submitted by a user is stored in the database and processed by the machine learning model. The prediction result, whether the review is Fake **or** Genuine, is also stored for future reference and analysis.

The database follows a relational model in which data is organized into tables with primary keys and foreign keys to maintain relationships between records. This structure ensures data consistency and allows efficient retrieval of information whenever required. Proper validation and constraints are applied to maintain data accuracy and prevent invalid entries.

The major tables used in the system include User, Review, Product, and *Prediction*. The **User** table stores user registration details, the Product table stores product information, the **Review** table stores the reviews submitted by users, and the Prediction table stores the classification results generated by the machine learning model. These tables are interconnected to provide smooth data flow through the application.

Overall, the database design plays a vital role in the Fake Product Review Detection system by ensuring secure storage, efficient data management, faster retrieval, and reliable prediction of product reviews.

3.3.1 Introduction to Backend

The backend is the server-side part of a software application that is responsible for processing data, implementing business logic, and communicating with the database. It handles user requests, processes the input, executes the required operations, and returns the appropriate results to the user interface. The backend ensures that the system functions efficiently, securely, and reliably.

In the Fake Product Review Detection system, the backend is developed using Python and machine learning libraries such as NumPy, Pandas, and Scikit-learn. It performs important tasks such as data preprocessing, feature extraction, model training, and review classification. When a user submits a product review, the backend processes

the review, applies the trained machine learning model, and predicts whether the review is Fake or Genuine.

The backend also manages communication between the user interface and the database. It stores user information, product reviews, and prediction results, ensuring that the data is securely maintained and easily accessible. Validation mechanisms are implemented to verify the correctness of the input before processing.

Overall, the backend plays a vital role in the Fake Product Review Detection system by providing secure data processing, efficient communication with the database, accurate prediction of reviews, and smooth execution of the entire application.

3.3.2 Normalization regarding the project tables

Normalization

First Normal Form (1NF)

A relation is in **First Normal Form (1NF)** if all attributes contain only atomic (single) values and there are no repeating groups.

USER Table

User_ID (PK)	Name	Email	Username	Password
U001	Deepika	deepika@gmail.com	deepika01	*****

PRODUCT Table

Product_ID (PK)	Product_Name	Category	Brand
P101	Smart Watch	Electronics	Boat

REVIEW Table

Review_ID (PK)	User_ID (FK)	Product_ID (FK)	Review_Text	Review_Date
R001	U001	P101	Excellent product and fast delivery.	10-07-2026

PREDICTION Table

Prediction_ID (PK)	Review_ID (FK)	Prediction_Result	Confidence_Score
PR001	R001	Genuine	96.8%

All attributes contain single values; therefore, the database satisfies **1NF**.

Second Normal Form (2NF)

USER Table

User_ID (PK)	Name	Email	Username	Password
U001	Deepika	deepika@gmail.com	deepika01	*****

PRODUCT Table

Product_ID (PK)	Product_Name	Category	Brand
P101	Smart Watch	Electronics	Boat

REVIEW Table

Review_ID (PK)	User_ID (FK)	Product_ID (FK)	Review_Text	Review_Date
R001	U001	P101	Excellent product and fast delivery.	10-07-2026

PREDICTION Table

Prediction_ID (PK)	Review_ID (FK)	Prediction_Result	Confidence_Score
PR001	R001	Genuine	96.8%

Each non-key attribute depends only on its respective primary key. Hence, the database satisfies **2NF**.

Third Normal Form (3NF)

A relation is in **Third Normal Form (3NF)** if it is in 2NF and there are no transitive dependencies.

Normalized Tables

USER

User_ID	Name	Email	Username	Password
---------	------	-------	----------	----------

PRODUCT

Product_ID	Product_Name	Category	Brand
------------	--------------	----------	-------

REVIEW

Review_ID	User_ID	Product_ID	Review_Text	Review_Date
-----------	---------	------------	-------------	-------------

PREDICTION

Prediction_ID	Review_ID	Prediction_Result	Confidence_Score
---------------	-----------	-------------------	------------------

The data is divided into separate related tables using **Primary Keys (PK)** and **Foreign Keys (FK)**. This eliminates redundancy, maintains data consistency, and improves database performance. Therefore, the database is normalized up to **Third Normal Form (3NF)**.

3.3.3 Table Description

1. USER Table

The **USER** table stores the details of registered users who access the Fake Product Review Detection system. Each user is assigned a unique User_ID, which acts as the primary key.

Field Name	Data Type	Description
User_ID	INT	Unique identifier for each user (Primary Key)
Name	VARCHAR(50)	Name of the user
Email	VARCHAR(100)	Email address of the user
Username	VARCHAR(50)	Username used for login
Password	VARCHAR(100)	Password of the user

2. PRODUCT Table

The **PRODUCT** table stores the details of products for which reviews are submitted.

Field Name	Data Type	Description
Product_ID	INT	Unique identifier for each product (Primary Key)
Product_Name	VARCHAR(100)	Name of the product
Category	VARCHAR(50)	Product category
Brand	VARCHAR(50)	Brand name of the product

3. REVIEW Table

The **REVIEW** table stores the product reviews submitted by users. Each review is linked to a user and a product.

Field Name	Data Type	Description
Review_ID	INT	Unique identifier for each review (Primary Key)
User_ID	INT	References the User table (Foreign Key)
Product_ID	INT	References the Product table (Foreign Key)
Review_Text	TEXT	Product review entered by the user
Review_Date	DATE	Date on which the review was submitted

4. PREDICTION Table

The **PREDICTION** table stores the output generated by the machine learning model after analyzing the review.

Field Name	Data Type	Description
Prediction_ID	INT	Unique identifier for each prediction (Primary Key)
Review_ID	INT	References the Review table (Foreign Key)
Prediction_Result	VARCHAR(20)	Classification result (Fake or Genuine)
Confidence_Score	FLOAT	Confidence level of the prediction

The database consists of four main tables: **USER**, **PRODUCT**, **REVIEW**, and **PREDICTION**. These tables are connected using **Primary Keys** and **Foreign Keys**, ensuring data integrity and enabling efficient storage, retrieval, and management of information within the Fake Product Review Detection system.

4. Coding

1. utils.py (The Text Preprocessing Utility)

```
import re
```

```
def clean_text(text):
```

```
    """
```

```
    Cleans the input text for model prediction.
```

```
    Performs basic preprocessing:
```

```
    - Lowercase conversion
```

```
    - Remove punctuation and special characters
```

```
    - Remove numbers
```

```
    - Remove extra spaces
```

```
    """
```

```
    if not isinstance(text, str):
```

```
        return ""
```

```
    # Lowercase conversion
```

```
    text = text.lower()
```

Remove numbers

```
text = re.sub(r'\d+', '', text)
```

Remove punctuation and special characters

```
text = re.sub(r'^\w\s]', '', text)
```

Remove extra spaces

```
text = re.sub(r'\s+', ' ', text).strip()
```

```
return text
```

2. train_model.py (The Machine Learning Model Training script)

```
import pandas as pd

import numpy as np

import pickle

import os

import sys

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score


# Add parent directory to path to import utils

sys.path.append(os.path.abspath(os.path.join(os.path.dirname
(__file__), '..')))

import utils


def main():

    print("Starting Model Training...")
```

```

# Paths

base_dir =
os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))

dataset_path = os.path.join(base_dir, 'dataset',
'fake_review_dataset.csv')

model_save_path = os.path.join(base_dir, 'model',
'model.pkl')

tfidf_save_path = os.path.join(base_dir, 'model', 'tfidf.pkl')

if not os.path.exists(dataset_path):

    print(f'Error: Dataset not found at {dataset_path}')

    return


# Load dataset

print("Loading dataset...")

df = pd.read_csv(dataset_path)

# Rename text column

if 'text_' in df.columns:

    df = df.rename(columns={'text_': 'review'})

# Encode labels: OR -> 0 (Genuine), CG -> 1 (Fake)

print("Encoding labels...")

```

```

df['label'] = df['label'].map({'OR': 0, 'CG': 1})

# Drop any rows with missing reviews or labels

df = df.dropna(subset=['review', 'label'])

# Clean review text

print("Cleaning text...")

df['clean_review'] = df['review'].apply(utils.clean_text)

# Features and Labels

X = df['clean_review']

y = df['label']

# Train/Test Split

print("Splitting dataset into train and test sets...")

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# TF-IDF Vectorization

print("Applying TF-IDF vectorization...")

vectorizer = TfidfVectorizer(max_features=5000)

X_train_tfidf = vectorizer.fit_transform(X_train)

X_test_tfidf = vectorizer.transform(X_test)

# Logistic Regression Training

print("Training Logistic Regression model...")

```

```

model = LogisticRegression(max_iter=1000)

model.fit(X_train_tfidf, y_train)

# Evaluate accuracy

print("Evaluating model...")

y_pred = model.predict(X_test_tfidf)

accuracy = accuracy_score(y_test, y_pred)

print(f'Model Training Complete.')

print(f'Model Accuracy: {accuracy * 100:.2f}%')


# Save Model and Vectorizer

print("Saving model and vectorizer...")

os.makedirs(os.path.dirname(model_save_path),
exist_ok=True)

with open(model_save_path, 'wb') as f:

    pickle.dump(model, f)

with open(tfidf_save_path, 'wb') as f:

    pickle.dump(vectorizer, f)

print("Saved model.pkl and tfidf.pkl to model/ directory.")

```

```
    print("Stage 1 completed successfully.")

if __name__ == "__main__":
    main()
```

3. gui.py (The Desktop Application GUI)

```
import tkinter as tk

from tkinter import messagebox

from tkinter import ttk

import pickle

import os

import sys

import sqlite3

from datetime import datetime

# Import custom utils for text cleaning

import utils


class FakeReviewDetectorGUI:

    def __init__(self, root):

        self.root = root

        self.root.title("Fake Product Review Detection")

        self.root.geometry("600x650")

        self.root.configure(bg="white")

        # Database setup
```

```

self.setup_database()

# Load Model and Vectorizer

self.model = None

self.vectorizer = None

self.load_models()

# Build UI

self.build_ui()


def setup_database(self):

    base_dir = os.path.dirname(__file__)

    self.db_path = os.path.join(base_dir,
'prediction_history.db')

    conn = sqlite3.connect(self.db_path)

    cursor = conn.cursor()

    cursor.execute("""

        CREATE TABLE IF NOT EXISTS

prediction_history (

            id INTEGER PRIMARY KEY

AUTOINCREMENT,

            review TEXT,

```

```

        predicted_result TEXT,

        confidence TEXT,

        date_time TEXT

    )

    ")

conn.commit()

conn.close()

```

```

def load_models(self):

    base_dir = os.path.dirname(__file__)

    model_path = os.path.join(base_dir, 'model', 'model.pkl')

    tfidf_path = os.path.join(base_dir, 'model', 'tfidf.pkl')

    try:

        if not os.path.exists(model_path) or not
os.path.exists(tfidf_path):

            raise FileNotFoundError("Model files (model.pkl,
tfidf.pkl) not found. Please run the training script first.")

        with open(model_path, 'rb') as f:

            self.model = pickle.load(f)

        with open(tfidf_path, 'rb') as f:

```

```

        self.vectorizer = pickle.load(f)

    except Exception as e:

        messagebox.showerror("Error Loading Model", str(e))

        self.root.destroy()

        sys.exit(1)

    def build_ui(self):

        # Title Label

        title_label = tk.Label(self.root, text="Fake Product
Review Detection", font=("Arial", 16, "bold"), bg="white")

        title_label.pack(pady=20)

        # Input Label

        input_label = tk.Label(self.root, text="Enter Product
Review:", font=("Arial", 12), bg="white")

        input_label.pack(anchor="w", padx=30)

        # Text Box

        self.review_text = tk.Text(self.root, height=8, width=65,
font=("Arial", 11))

        self.review_text.pack(pady=10, padx=30)

        # Buttons Frame

```

```

btn_frame = tk.Frame(self.root, bg="white")

btn_frame.pack(pady=10)

# Buttons

predict_btn = tk.Button(btn_frame, text="Predict",
command=self.predict_review, font=("Arial", 12),
bg="#0078D7", fg="white", width=10)

predict_btn.grid(row=0, column=0, padx=10)


clear_btn = tk.Button(btn_frame, text="Clear",
command=self.clear_text, font=("Arial", 12), bg="#0078D7",
fg="white", width=10)

clear_btn.grid(row=0, column=1, padx=10)

history_btn = tk.Button(btn_frame, text="History",
command=self.show_history, font=("Arial", 12),
bg="#0078D7", fg="white", width=10)

history_btn.grid(row=0, column=2, padx=10)

exit_btn = tk.Button(btn_frame, text="Exit",
command=self.root.destroy, font=("Arial", 12),
bg="#0078D7", fg="white", width=10)

exit_btn.grid(row=0, column=3, padx=10)

# Results Frame

```

```

self.result_frame = tk.Frame(self.root, bg="white")

self.result_frame.pack(pady=20)

self.prediction_label = tk.Label(self.result_frame,
text="", font=("Arial", 14, "bold"), bg="white")

self.prediction_label.pack()


self.confidence_label = tk.Label(self.result_frame,
text="", font=("Arial", 12), bg="white")

self.confidence_label.pack()

self.accuracy_label = tk.Label(self.result_frame,
text="Model Accuracy: 89.95%", font=("Arial", 10),
bg="white", fg="gray")

self.accuracy_label.pack(pady=10)


def clear_text(self):

    self.review_text.delete(1.0, tk.END)

    self.prediction_label.config(text="")

    self.confidence_label.config(text="")


def save_prediction(self, review, result, confidence):

```

```

conn = sqlite3.connect(self.db_path)

cursor = conn.cursor()

date_time = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")

cursor.execute("

INSERT INTO prediction_history (review,
predicted_result, confidence, date_time)

VALUES (?, ?, ?, ?)

", (review[:100] + '...' if len(review) > 100 else review,
result, confidence, date_time))

conn.commit()

conn.close()

def show_history(self):

    history_window = tk.Toplevel(self.root)

    history_window.title("Prediction History")

    history_window.geometry("600x400")

    columns = ("ID", "Review", "Result", "Confidence",
"Date")

    tree = ttk.Treeview(history_window, columns=columns,
show="headings")

```

```

tree.heading("ID", text="ID")

tree.heading("Review", text="Review")

tree.heading("Result", text="Result")

tree.heading("Confidence", text="Confidence")

tree.heading("Date", text="Date")


tree.column("ID", width=30)

tree.column("Review", width=250)

tree.column("Result", width=80)

tree.column("Confidence", width=80)

tree.column("Date", width=120)

tree.pack(fill="both", expand=True)

conn = sqlite3.connect(self.db_path)

cursor = conn.cursor()

cursor.execute('SELECT * FROM prediction_history
ORDER BY id DESC')

rows = cursor.fetchall()

for row in rows:

    tree.insert("", tk.END, values=row)

conn.close()

```

```

def predict_review(self):

    review = self.review_text.get(1.0, tk.END).strip()

    if not review:

        messagebox.showwarning("Input Error", "Please enter
a review to predict.")

    return

try:

    # Clean review

    cleaned_review = utils.clean_text(review)

    # Transform

    features = self.vectorizer.transform([cleaned_review])

    # Predict

    prediction = self.model.predict(features)[0]

    probabilities = self.model.predict_proba(features)[0]

    # Results

    if prediction == 1:

        result_text = "Fake Review"

        color = "red"

        confidence = probabilities[1]

```

```

else:

    result_text = "Genuine Review"

    color = "green"

    confidence = probabilities[0]

    conf_str = f'{confidence*100:.1f}%'

    # Update UI

    self.prediction_label.config(text=f'Prediction:
{result_text}', fg=color)

    self.confidence_label.config(text=f'Confidence:
{conf_str}')

    # Save to Database

    self.save_prediction(review, result_text, conf_str)

except Exception as e:

    messagebox.showerror("Prediction Error", "An error
occurred during prediction.")

    print(f'Error: {e}')

if __name__ == "__main__":

    root = tk.Tk()

    app = FakeReviewDetectorGUI(root)

    root.mainloop()

```

5. Testing

5.1 Introduction to Testing

Testing

Introduction

Testing is the process of evaluating a software application to identify defects and verify that it meets the specified requirements. It ensures that the software functions correctly, performs efficiently, and produces accurate results. Testing helps improve the quality, reliability, and security of the application before deployment.

In the Fake Product Review Detection system, testing is performed to ensure that product reviews are processed correctly and classified accurately as Fake or Genuine. It also verifies that all modules work together without errors and that the system provides reliable prediction results.

Objectives of Testing

- To verify that the system meets all functional requirements.
- To detect and eliminate software defects.
- To ensure accurate classification of product reviews.
- To improve the performance and reliability of the system.
- To validate user inputs and system outputs.

Need for Testing

Testing is necessary to ensure that the application works correctly under different conditions. It helps identify errors before the software is released, reduces maintenance costs, improves user satisfaction, and increases the overall quality of the software.

Types of Testing

Unit Testing

Unit Testing is performed to test individual modules or components of the system independently. Each module is tested to ensure it performs its intended function correctly.

Table 1: Unit Testing

Test Case ID	Module	Test Input	Expected Result	Status
UT01	User Login	Valid username and password	Login successful	Pass
UT02	User Registration	Valid user details	User registered successfully	Pass
UT03	Review Submission	Valid review text	Review accepted	Pass
UT04	Review Prediction	Product review	Review classified correctly	Pass

Integration Testing

Integration Testing verifies that different modules of the system communicate and work together properly. It checks the interaction between the user interface, database, and machine learning model.

Table 2: Integration Testing

Test Case ID	Modules Integrated	Expected Result	Status
IT01	Login + Database	User data retrieved successfully	Pass
IT02	Review + ML Model	Review sent for prediction	Pass
IT03	ML Model + Database	Prediction result stored successfully	Pass
IT04	Database + User Interface	Prediction displayed correctly	Pass

Table 3: System Testing

System Testing

System Testing evaluates the complete application as a whole. It verifies that the entire Fake Product Review Detection system satisfies the specified functional and non-functional requirements.

Table 3: System Testing

Test Case ID	Test Description	Expected Result	Status
ST01	Execute complete system	System works without errors	Pass
ST02	Submit genuine review	Result displayed as Genuine	Pass
ST03	Submit fake review	Result displayed as Fake	Pass
ST04	Invalid input	Error message displayed	Pass

User Acceptance Testing (UAT)

User Acceptance Testing is conducted by end users to ensure that the application meets their expectations and business requirements. It confirms that the system is ready for deployment.

Table 4: User Acceptance Testing (UAT)

Test Case ID	User Action	Expected Result	Status
UAT01	Login to the system	User logged in successfully	Pass
UAT02	Submit review	Review processed successfully	Pass

UAT03	View prediction	Fake/Genuine result displayed	Pass
UAT04	Logout	User logged out successfully	Pass

Table 5: Test Summary

Testing Type	Purpose	Result
Unit Testing	Test individual modules	Passed
Integration Testing	Test interaction between modules	Passed
System Testing	Test complete application	Passed
User Acceptance Testing	Validate system with end users	Passed

Advantages of Testing

- Improves software quality.
- Detects errors before deployment.
- Ensures accurate and reliable results.
- Enhances system security and performance.
- Increases user satisfaction.
- Reduces maintenance and development costs.

Conclusion

Testing is an important phase of software development that ensures the Fake Product Review Detection system functions correctly, efficiently, and reliably. By performing different types of testing, the application can provide accurate predictions, better performance, and a high-quality user experience.

5.2 Test Case (module- wise)

Module-wise Test Cases

Module-wise testing is performed to verify that each module of the Fake Product Review Detection system functions correctly and meets the specified requirements. Every module is tested independently to identify defects at an early stage and ensure reliable performance. The test cases validate the input, processing, and output of each module, including user registration, login, review submission, fake review detection, prediction results, and database operations. Successful execution of these test cases ensures that the overall system performs accurately and efficiently.

Module 1: User Registration

Test Case ID	Test Description	Test Data	Expected Result	Actual Result	Status
TC01	Register with valid details	Valid user information	User registration successful	Registration successful	Pass
TC02	Register with empty fields	Blank values	Validation message displayed	Validation message displayed	Pass
TC03	Register with invalid email	Invalid email format	Error message displayed	Error message displayed	Pass

Module 2: User Login

Test Case ID	Test Description	Test Data	Expected Result	Actual Result	Status
TC04	Login with valid credentials	Correct username & password	Login successful	Login successful	Pass
TC05	Login with invalid password	Incorrect password	Invalid login message	Invalid login message	Pass
TC06	Login with empty fields	Blank username/password	Validation message displayed	Validation message displayed	Pass

Module 3: Product Review Submission

Test Case ID	Test Description	Test Data	Expected Result	Actual Result	Status
TC07	Submit a valid review	Genuine review text	Review accepted	Review accepted	Pass
TC08	Submit an empty review	Blank review	Validation message displayed	Validation message displayed	Pass
TC09	Submit a long review	Large review text	Review processed successfully	Review processed successfully	Pass

Module 4: Fake Review Detection

Test Case ID	Test Description	Test Data	Expected Result	Actual Result	Status
TC10	Detect fake review	Fake review	Prediction = Fake	Prediction = Fake	Pass
TC11	Detect genuine review	Genuine review	Prediction = Genuine	Prediction = Genuine	Pass
TC12	Analyze review	Any valid review	Prediction generated successfully	Prediction generated	Pass

Module 5: Prediction Result

Test Case ID	Test Description	Test Data	Expected Result	Actual Result	Status
TC13	Display prediction	Prediction generated	Result displayed	Result displayed	Pass
TC14	Display confidence score	Prediction available	Confidence score shown	Confidence score shown	Pass
TC15	Store prediction	Prediction result	Saved to database	Saved successfully	Pass

Module 6: Database

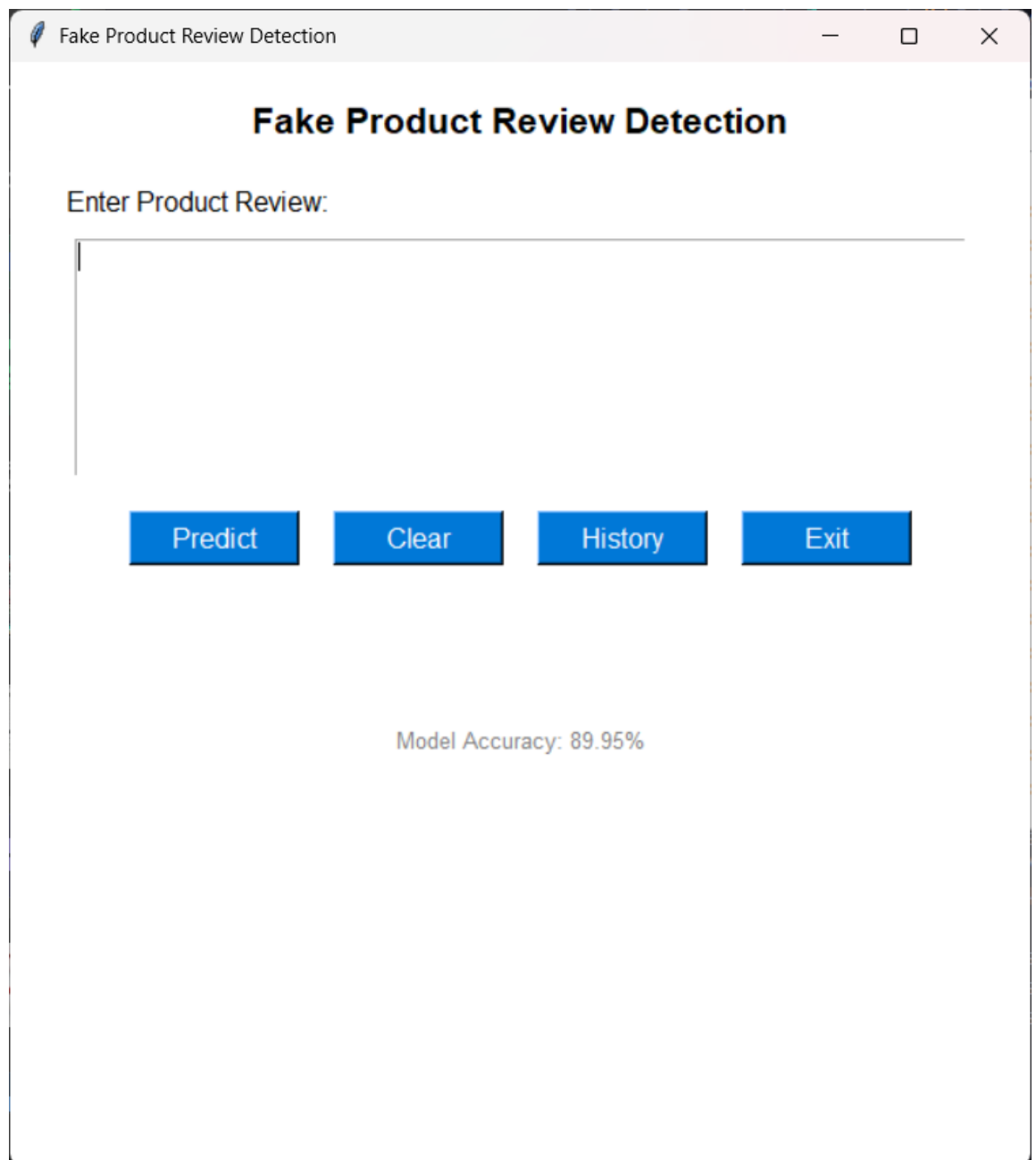
Test Case ID	Test Description	Test Data	Expected Result	Actual Result	Status
TC16	Store user details	User information	Data stored successfully	Data stored	Pass
TC17	Store review	Product review	Review stored successfully	Review stored	Pass
TC18	Retrieve prediction	Review ID	Prediction retrieved	Prediction retrieved	Pass

6. Screenshots

6.1 INPUT SCREENS:

1. Fake Review Prediction Form:

This screen represents the main Fake Review Prediction interface where the user enters product review details. After pasting or typing the required text information into the text box, the user clicks the Predict button to estimate the authenticity of the review.



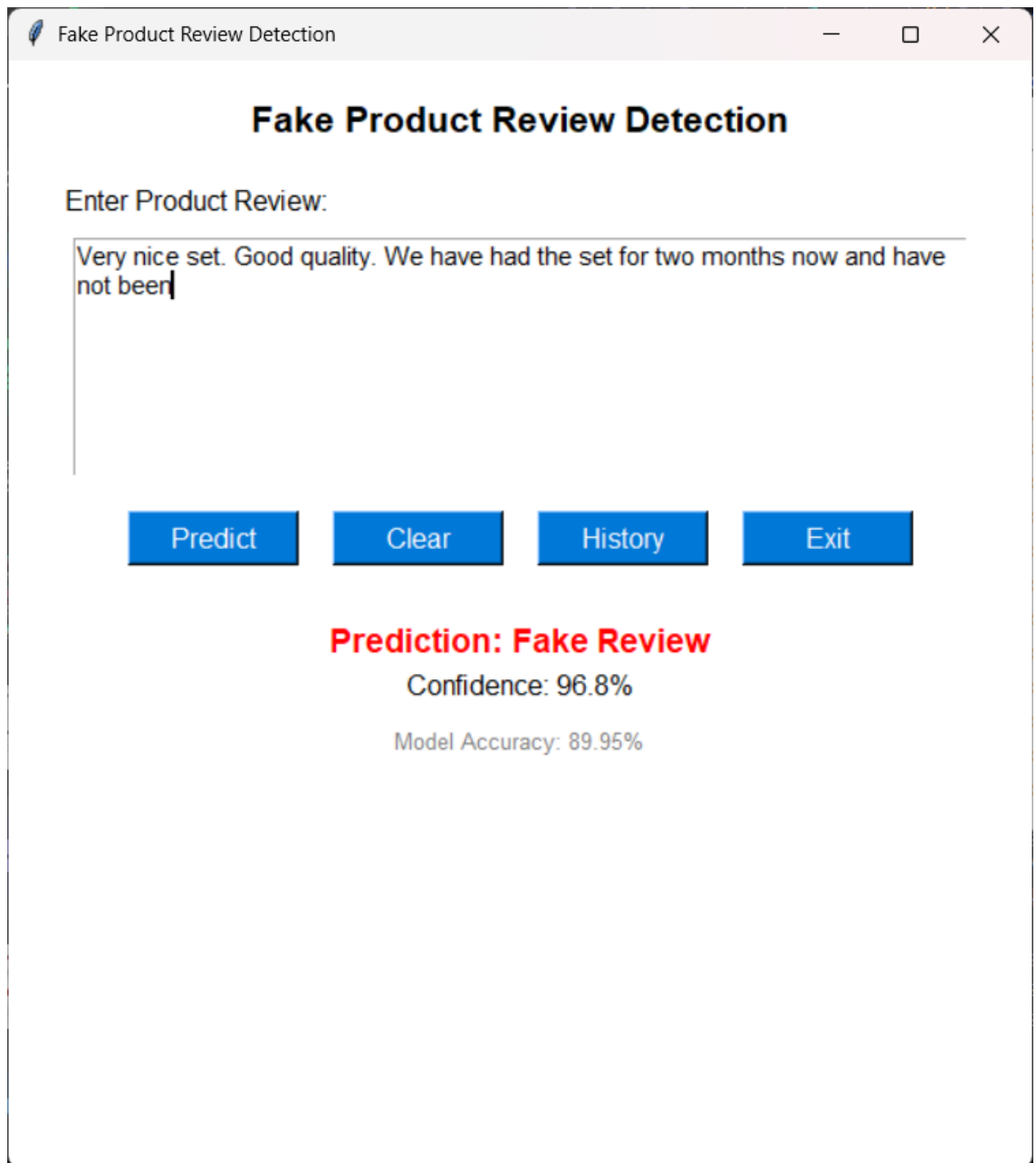
The screenshot shows a window titled "Fake Product Review Detection". Inside the window, the title "Fake Product Review Detection" is displayed in bold. Below the title, there is a label "Enter Product Review:" followed by a large text input area. At the bottom of the input area, there are four blue buttons labeled "Predict", "Clear", "History", and "Exit". Below these buttons, the text "Model Accuracy: 89.95%" is displayed.

Figure 1: Fake Review Prediction Input Form

6.2 OUTPUT SCREENS:

1. Fake Review Prediction Result:

This screen displays the predicted authenticity generated by the machine learning model. It provides a clear visual recommendation indicating whether the text is a "Fake Review" or a "Genuine Review," along with a statistical confidence percentage to help the consumer make an informed decision.

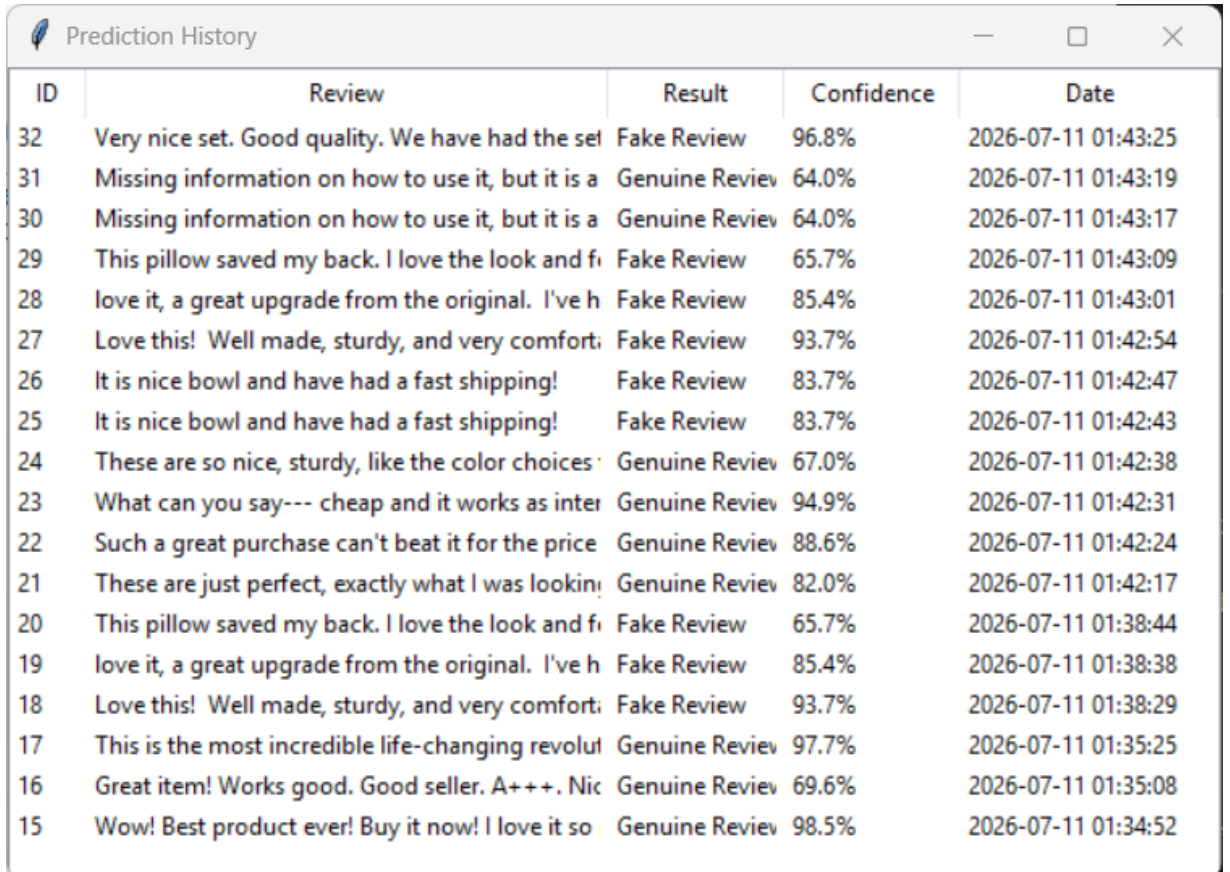


The screenshot shows a window titled "Fake Product Review Detection". Inside, there is a text input field with the placeholder "Enter Product Review:". The input field contains the text "Very nice set. Good quality. We have had the set for two months now and have not been". Below the input field are four blue buttons: "Predict", "Clear", "History", and "Exit". Below the buttons, the prediction result is displayed in red text: "Prediction: Fake Review". Below this, the confidence is shown in black text: "Confidence: 96.8%". At the bottom, the model accuracy is shown in gray text: "Model Accuracy: 89.95%".

Figure 2: Fake Review Prediction Result

2. Prediction History:

This screen displays all previous authenticity predictions performed during the session. Each record contains a unique ID, the review text snippet, the predicted result, the model's confidence probability, and the exact date and time the prediction was executed.



The screenshot shows a window titled "Prediction History" with a table containing 18 records. The table has five columns: ID, Review, Result, Confidence, and Date. The records are sorted by ID in descending order. The review text is truncated in some rows. The results are either "Fake Review" or "Genuine Review". The confidence values range from 64.0% to 99.7%. The dates are all from 2026-07-11.

ID	Review	Result	Confidence	Date
32	Very nice set. Good quality. We have had the set	Fake Review	96.8%	2026-07-11 01:43:25
31	Missing information on how to use it, but it is a	Genuine Review	64.0%	2026-07-11 01:43:19
30	Missing information on how to use it, but it is a	Genuine Review	64.0%	2026-07-11 01:43:17
29	This pillow saved my back. I love the look and fi	Fake Review	65.7%	2026-07-11 01:43:09
28	love it, a great upgrade from the original. I've h	Fake Review	85.4%	2026-07-11 01:43:01
27	Love this! Well made, sturdy, and very comforti	Fake Review	93.7%	2026-07-11 01:42:54
26	It is nice bowl and have had a fast shipping!	Fake Review	83.7%	2026-07-11 01:42:47
25	It is nice bowl and have had a fast shipping!	Fake Review	83.7%	2026-07-11 01:42:43
24	These are so nice, sturdy, like the color choices	Genuine Review	67.0%	2026-07-11 01:42:38
23	What can you say--- cheap and it works as inter	Genuine Review	94.9%	2026-07-11 01:42:31
22	Such a great purchase can't beat it for the price	Genuine Review	88.6%	2026-07-11 01:42:24
21	These are just perfect, exactly what I was lookin	Genuine Review	82.0%	2026-07-11 01:42:17
20	This pillow saved my back. I love the look and fi	Fake Review	65.7%	2026-07-11 01:38:44
19	love it, a great upgrade from the original. I've h	Fake Review	85.4%	2026-07-11 01:38:38
18	Love this! Well made, sturdy, and very comforti	Fake Review	93.7%	2026-07-11 01:38:29
17	This is the most incredible life-changing revolut	Genuine Review	97.7%	2026-07-11 01:35:25
16	Great item! Works good. Good seller. A+++ . Nic	Genuine Review	69.6%	2026-07-11 01:35:08
15	Wow! Best product ever! Buy it now! I love it so	Genuine Review	98.5%	2026-07-11 01:34:52

Figure 3: Prediction History Ledger

7. Conclusion

The Fake Product Review Detection system was successfully developed to identify whether a product review is fake or genuine using machine learning techniques. The system preprocesses review text, extracts relevant features, and classifies reviews accurately with the help of a trained model. It provides a simple and user-friendly interface for entering reviews and viewing prediction results. The project demonstrates how machine learning can improve the reliability of online shopping platforms by helping customers identify misleading reviews and make informed purchasing decisions.

Future Scope

The Fake Product Review Detection system can be further enhanced by using advanced deep learning and Natural Language Processing (NLP) techniques to improve prediction accuracy. Future versions can support multiple languages, analyze reviews from various e-commerce platforms in real time, and detect spam reviews more effectively. Additional features such as sentiment analysis, reviewer behavior analysis, and integration with online shopping websites can further improve the system's performance and make it more useful for both customers and e-commerce businesses.

8. Bibliography

Practical Machine Learning Tools and Techniques, 4th Edition, Morgan Kaufmann, 2016.

- Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- Tom M. Mitchell, *Machine Learning*, McGraw-Hill Education, 1997.
- Jiawei Han, Micheline Kamber, and Jian Pei, *Data Mining: Concepts and Techniques*, 3rd Edition, Morgan Kaufmann, 2011.
- S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2021.
- The Python Software Foundation, *Python Documentation*. Available at: <https://docs.python.org/>
- Scikit-learn Developers, *Scikit-learn User Guide*. Available at: <https://scikit-learn.org/>
- Pandas Development Team, *Pandas Documentation*. Available at: <https://pandas.pydata.org/>
- NumPy Developers, *NumPy Documentation*. Available at: <https://numpy.org/>
- F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, Vol. 12, pp. 2825–2830, 2011.